

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



APPLICATIONS OF NATURAL LANGUAGE PROCESSING IN INDUSTRY

Project 4

REST, gRPC AND DEPLOYMENT WITH DOCKER

Lecturer: Dr. Nguyen Hong Buu Long

Ho Chi Minh City, 12th May 2026

Contents

1	Project Description	2
2	Task Requirements	3
2.1	System Architecture	3
2.2	API Gateway	4
2.3	Intent Service with gRPC	4
2.4	Response Generation with Ollama	5
2.5	Docker Containerization	5
2.6	Multi-container Deployment with Docker Compose	6
2.7	README Requirements	6
2.8	Video Demonstration	7
2.9	Project Structure	8
2.9.1	Banking-service	8
2.9.2	Back-end	8
2.9.3	Front-end	9
2.9.4	Intent-service	9



1 Project Description

In this lab, students are required to transform a banking AI agentic system from a local prototype into a **multi-service deployment architecture**.

The main objective of this project is to help students understand how to design, containerize, and deploy an AI-powered application as a modular service-based system. Instead of running all components in a single local program, students will separate the system into multiple services that communicate with each other through HTTP and gRPC.

The deployed system must include the following main services:

- An **API Gateway** that receives external HTTP requests and coordinates the agentic workflow.
- An **Intent Service** implemented as an independent **gRPC microservice** for intent detection.
- An **Ollama Service** used to serve the response generation model.
- Additional workflow components such as priority detection, policy retrieval, validation, and routing.

Students must redesign the original agentic workflow so that it can run as a reproducible multi-container application. The API Gateway will act as the main entry point of the system. It receives customer messages, calls the Intent Service through gRPC, executes the remaining workflow nodes, calls the response generation model through Ollama, and returns the final structured output.

2 Task Requirements

2.1 System Architecture

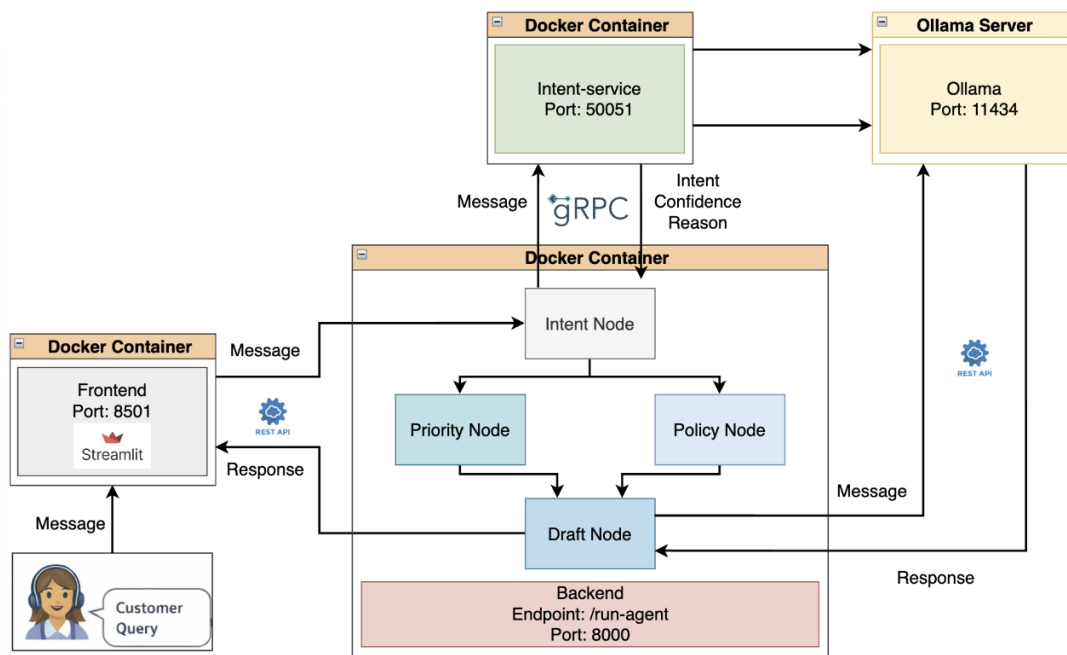


Figure 1: An overview of the architecture and used tools

Attention: Students are encouraged to refer to the template files in the folder **Documents** on Moodle for more details, including the following:

- **Ollama.ipynb**
- **intent-service.proto**
- **Dockerfile:**
- **docker-compose.yml**
- **Makefile**

Students must design the project as a **microservice-based system**. The system should contain at least the following components:

- **API Gateway:** receives requests from users or external clients and orchestrates the full workflow.
- **Intent Service:** a separate gRPC microservice responsible for predicting the intent of a customer message.
- **Ollama Service:** serves the language model used for response generation.
- **Frontend:** provides a simple interface for users to interact with the system.

The API Gateway must communicate with the Intent Service using gRPC. It must also communicate with the Ollama service through HTTP to generate responses.

2.2 API Gateway

Students must implement the API Gateway using **FastAPI**. The API Gateway must provide the following endpoints:

- GET `/health`: check whether the system is running.
- GET `/config`: return the current system configuration.
- POST `/run-agent`: execute the full agentic workflow.

The API Gateway is responsible for:

- Receiving customer messages.
- Calling the Intent Service through gRPC.
- Executing additional workflow nodes.
- Calling the response generation model through Ollama.
- Returning the final structured result.

2.3 Intent Service with gRPC

Students must implement the intent detection component as an independent **gRPC microservice**. The service must expose a gRPC method for intent prediction. The input of this method should be a customer message, and the output should include at least:



- The predicted intent.
- The confidence score.

Students may use the fine-tuned model from **Lab 2** or use **Ollama** to perform intent prediction. Students must also define a `.proto` file and generate the corresponding gRPC client and server code.

2.4 Response Generation with Ollama

The response generation node must call a model served through **Ollama**. Students are encouraged to use **GPT-OSS** as the response generation model.

The response generation node should use the following information:

- The customer message.
- The predicted intent.
- Priority or risk information.
- Retrieved policy content.

The output of this node should include:

- A draft reply.
- Missing information, if any.
- The next recommended action.

2.5 Docker Containerization

Students must write Dockerfiles for the required services, including:

- The API Gateway.
- The Intent Service.
- The Frontend.

Each service must be containerized so that it can run independently and reproducibly.

2.6 Multi-container Deployment with Docker Compose

Students must create a `docker-compose.yml` file to run the full system.

The Docker Compose configuration should include at least:

- `api-gateway`.
- `intent-service`.
- `frontend`.

The Docker Compose file must define:

- Service names.
- Exposed ports.
- Environment variables.
- Inter-service dependencies.

The API Gateway must be able to communicate with:

- The Intent Service through gRPC.
- The Ollama Service through HTTP.

2.7 README Requirements

The project README must include:

- A description of the microservice architecture.
- Instructions for generating gRPC code from the `.proto` file.
- Instructions for building Docker images.
- Instructions for running the system with Docker Compose.
- A description of the role of each container.



2.8 Video Demonstration

Students must submit a short video demonstration of the completed system. The video should show:

- Introducing an overview system.
- Running the system using `docker-compose`.
- Calling the API Gateway,
- Showing the final workflow output.

The recommended video duration is **2–3 minutes**.

2.9 Project Structure

2.9.1 Banking-service

```
banking-service
├── backend
├── frontend
├── intent_service
└── docker-compose.yml
```

2.9.2 Back-end

```
banking-service
├── backend
│   └── app
│       ├── agent
│       │   └── orchestrator.py
│       ├── clients
│       │   ├── base.py
│       │   ├── grpc_intent_client.py
│       │   ├── intent_grpc
│       │   │   ├── intent_service_pb2_grpc.py
│       │   │   └── intent_service_pb2.py
│       │   └── ollama_client.py
│       ├── core
│       │   ├── schemas.py
│       │   └── settings.py
│       ├── data
│       │   └── policies.py
│       ├── main.py
│       └── nodes
│           ├── draft_node.py
│           ├── intent_node.py
│           ├── policy_node.py
│           ├── priority_node.py
│           ├── router_node.py
│           └── validation_node.py
├── Dockerfile
├── README.md
├── requirements.txt
├── run.py
└── docker-compose.yml
```

2.9.3 Front-end

```
banking-service
├── frontend
│   ├── Dockerfile
│   ├── interface.py
│   ├── requirements.txt
│   └── docker-compose.yml
```

2.9.4 Intent-service

```
banking-service
├── intent_service
│   ├── app
│   │   ├── clients
│   │   │   ├── base.py
│   │   │   └── ollama_client.py
│   │   ├── core
│   │   │   ├── schemas.py
│   │   │   └── settings.py
│   │   ├── data
│   │   │   └── policies.py
│   │   └── nodes
│   │       └── intent_node.py
│   ├── client.py
│   ├── Dockerfile
│   ├── intent_service_pb2_grpc.py
│   ├── intent_service_pb2.py
│   ├── intent_service.proto
│   ├── Makefile
│   ├── requirements.txt
│   ├── server.py
│   └── docker-compose.yml
```